# AI LOG :

**Task #28 : Generate Initial User Stories**

Purpose of AI use : To generate 10 user stories

Chat Link or Prompt : give me 10 user stories. File referenced named "SOEN341_CareerConnect_Project_Sprint1.docx

Response :
US-01 : As a job seeker, I want to register for an account and manage my profile so that I can securely access the platform's job search features.
US-02: As a job seeker, I want to upload and manage my resumes so that I have my documents readily available for different applications.
US-03: As a job seeker, I want to search and filter job postings by keywords so that I can find opportunities that match my skillset.
US-04: As a job seeker, I want to save and favourite specific job postings so that I can easily find and apply to them later.
US-05: As a job seeker, I want to submit my application directly through the platform so that all my job-hunting activities remain centralized.
US-06: As a job seeker, I want to view an application dashboard so that I can track the status of my submissions (Applied, Interview, Offered, Rejected) in one place.
US-07: As a job seeker, I want to receive notifications and reminders for application deadlines so that I do not miss out on target opportunities.
US-08: As a recruiter, I want to create and manage job postings so that I can advertise open roles to active job seekers.
US-09: As a recruiter, I want to view and filter submitted applications for my postings so that I can efficiently evaluate candidates and update their status.
US-10: As a job seeker, I want to receive AI-assisted feedback on my uploaded resume so that I can improve its quality before submitting an application.

AI-Suggested Content: Generated an initial list of 10 user stories covering account management, job search workflows, recruiter features, and the mandatory AI integration.

Validation : Compared the AI generated stories to the system features listed in the project sprint doc and it matches with the requirements.

Decision : Accepted by the team and modified

Reflection : The AI effectively translated the project requirements and we will use this as a foundation and go through our own brain storm session to generate original features.

Responsible Person : Zohair Ameeri

**Task #105 : Sprint 2 Requirements: Write and refine acceptance tests for user stories**

Purpose of AI use : To understand what acceptance tests are and how to implement them in our GitHub repository (as Given/When/Then scenarios in GitHub Issues), so I can refine the draft AT issues (#68, #69, #88, #89, #90, #91, #92, #93, #94, #95)

Chat Link or Prompt : Prompt: pasted the Sprint 2 Requirements task description (create at least two acceptance tests per Sprint 1 and Sprint 2 user story, stored as labelled GitHub issues linked to their story and tasks) and asked "what are acceptance tests and how do I implement them in my GitHub repo?"

Response : An explanation of acceptance tests as end-user scenarios written in Given/When/Then format, with a login example (one happy path, one error path). A Markdown template for AT issues. Steps to edit the draft issues, apply the acceptance-test label, link each AT issue to its user story as a sub-issue or through # cross-references, and check each AT against the story's acceptance criteria.

AI-Suggested Content : The Given/When/Then structure, the happy path plus error/edge case pattern for meeting the two-tests-per-story requirement, the AT issue Markdown template, the labelling and linking workflow, and tips for writing concrete, observable test wording.

Validation : I compared the AI's explanation and workflow with the Sprint 2 guidelines and the course material, and tested the template and linking steps for AT’s on the repo.

Decision : Accepted and modified according to the AT’s specification

Reflection : The AI gave a clear starting structure, but the scenarios still had to be written and checked against each story's real criteria.

Responsible Person : Zohair Ameeri

**Task #007 : Audit implemented frontend features vs backend-only features**

Purpose of AI use : To get an accurate list of the features the website actually has on the frontend, separate from what the backend and README describe, so the team's feature list reflects the real state of the project

Chat Link or Prompt : Prompt: what features does the website currently have, based on the actual frontend screens and backend routes rather than the README

Response : Right now, your website has these **visible frontend features**:

- **Login** with email and password
- **Registration** for new users
- Automatic login after registration
- JWT-based authentication
- Redirect to a personal profile page after login
- **Resume upload**
- View uploaded resumes
- Delete uploaded resumes
- Login error messages for invalid credentials or duplicate emails
- Protected profile access for logged-in users

The frontend currently has only two pages: `/login` and `/profile/:user`, as defined in `app.routes.ts`.

Your backend also supports features that are **not connected to the frontend yet**:

- Browse and search jobs by keyword, location, and category
- View individual jobs
- Recruiters creating job postings
- Applying to jobs
- Viewing submitted applications
- Recruiters updating application statuses
- Editing user profile information
- Replacing uploaded resumes
- Logout API endpoint

These backend capabilities are listed in `README.md`, but there are currently no corresponding Angular screens for jobs, applications, recruiter dashboards, or profile editing.

AI-Suggested Content : The split between visible frontend features and backend-only capabilities, and the finding that there are no Angular screens yet for jobs, applications, recruiter dashboards, or profile editing.

Validation : I checked it locally by manually testing the website.

Decision : Accepted to get a better understanding of workflow

Reflection : the AI gave a quick overview of frontend vs backend coverage, but it only reflects the code at that point and had to be re-checked against the repo.

Responsible Person : Zohair Ameeri

**Task #75: Frontend: Registration view with role choice, validation, and backend error messages (User story #38)**

Purpose of AI use : To check whether the Frontend registration task for user story #38 had been completed as my other teammate already implemented some features assigned to me in another issue. So by tracing the code against each requirement in the task description , I wanted to figure out if anything is left to complete.

Chat Link or Prompt : Prompt: pasted the task description (role choice sent as role to POST /auth/signup; client-side validation of email format and password of at least 8 characters with submit disabled until valid; show the backend message for 400 responses while keeping the 401/409 messages; land on the role's home page after sign-up) and asked "has this task been done, check for me".

Response : Status: Partially done — not complete

| Requirement | Status | Evidence |
|---|---|---|
| Role choice: Job seeker / Recruiter | ✅ Done | Role toggle exists in login.component.html, with job_seeker and recruiter options. |
| Send selected role to POST /auth/signup | ✅ Done | auth.service.ts sends { email, password, role }. |
| Client-side valid email validation | ✅ Done | login.component.ts validates the email before registration. |
| Password must be at least 8 characters | ❌ Missing | register() in login.component.ts does not validate password length. |
| Submit disabled until valid | ❌ Missing | The Register button in login.component.html has no [disabled] condition. |
| Show backend error message for 400 responses | ❌ Missing | Registration currently maps every non-409 error to "Something went wrong" in login.component.ts. The backend does return useful 400 messages such as "Valid email is required" and "Password must be at least 8 characters" from auth.js. |
| Preserve existing 401/409 messages | ✅ Mostly done | Existing wrong-password and duplicate-email messages remain in login.component.html. |

Redirect to role-specific home after signup ✅ Done login.component.ts redirects through redirectByRole(): recruiters go to /recruiter/postings, job seekers go to /jobs.

Therefore, the story is not fully completed. The missing work is password validation, disabling Register until the form is valid, and displaying backend 400 error messages.

AI-Suggested Content : The requirement-by-requirement status table (done, partial, missing) with the files that supported each finding (login.component.ts, login.component.html, auth.service.ts, and the backend auth route).

Validation : I opened the files the AI referenced and confirmed that password length validation, the disabled Register button and 400 message handling were not implemented, and tested registration with a short password in the running app.

Decision : Accepted the findings
Reflection : The AI quickly pinpointed which requirements were still missing, but the status only reflects the code at the time of the check and had to be confirmed in the repo and the running app.

Responsible Person : Zohair Ameeri

**Task #122 : Frontend: Login/register errors + password validation, and profile error/recruiter fixes (follows #74, #75)**

Purpose of AI use : To find out which files to work in and what to change in order to complete the remaining behaviours assigned to me in the review of PR #112 (login/register errors, password validation, profile save error, hiding Resumes for recruiters)

Chat Link or Prompt : Pasted my assignment from the PR #112 review (make the login error state a signal, show the backend's own message for 400s, add a client-side check that the password is at least 8 characters, show an error if saving the profile fails, hide Resumes for recruiters) and asked what I have to do to complete the missing behaviours, which files to work in, and what to change.

Response : I'll map each missing behavior to the exact frontend files and current code paths, including the profile component, so the change list is precise rather than speculative.

```
Read
Completed with input: {
```

```
  "path":
"C:\\Users\\zohai\\Downloads\\cloone\\SOEN-341-agile-project-group-back-row\\frontend\\CareerConnect\\src\\app\\profile.component.ts"
}
Read
Completed with input: {
  "path":
"C:\\Users\\zohai\\Downloads\\cloone\\SOEN-341-agile-project-group-back-row\\frontend\\CareerConnect\\src\\app\\profile.component.html"
}
Read
Completed with input: {
  "path":
"C:\\Users\\zohai\\Downloads\\cloone\\SOEN-341-agile-project-group-back-row\\frontend\\CareerConnect\\src\\app\\login.component.ts"
}
"Read" failed
Completed with input: {
  "path":
"C:\\Users\\zohai\\Downloads\\cloone\\SOEN-341-agile-project-group-group-back-row\\frontend\\CareerConnect\\src\\app\\login.component.html"
}
Tool execution failed: Path does not exist
To complete the missing behaviors, you mainly need to modify these files:
- `login.component.ts`
- `login.component.html`
- `profile.component.ts`
- `profile.component.html`
## 1. Make login/register errors reactive
### File: `login.component.ts`
Currently:
```ts
display_error = 0;
```
Because the application is zoneless, changing a normal field inside an HTTP callback does not reliably update the template.
Change it to a signal:
```ts
import { Component, inject, signal } from '@angular/core';
```
```

Then:
```ts
display_error = signal(0);
backend_error = signal('');
```
Every assignment must use `.set()`.
For example, change:
```ts
this.display_error = this.INVALID_EMAIL;
```
to:
```ts
this.display_error.set(this.INVALID_EMAIL);
```
And:
```ts
this.display_error = 0;
```
to:
```ts
this.display_error.set(0);
this.backend_error.set('');
```
### Preserve the existing 401 and 409 messages
For login:
```ts
error: (err: HttpErrorResponse) => {
  this.display_error.set(
    err.status === 401 ? this.WRONG_PASSWORD : this.OTHER_ERROR,
  );
},
```
For registration:
```ts
error: (err: HttpErrorResponse) => {
  if (err.status === 409) {
    this.display_error.set(this.USERNAME_TAKEN);
    return;
  }
  if (err.status === 400) {
```

```
    this.backend_error.set(err.error?.error ?? 'Invalid registration details');
    this.display_error.set(this.OTHER_ERROR);
    return;
  }
  this.display_error.set(this.OTHER_ERROR);
},
```
The backend returns errors in this format:
```json
{
  "error": "Password must be at least 8 characters"
}
```
So the important expression is:
```ts
err.error?.error
```
## 2. Add client-side password validation
### File: `login.component.ts`
Add a password error constant:
```ts
INVALID_PASSWORD = 5;
```
In `register()`, validate the password before calling the API:
```ts
if (password.length < 8) {
  this.display_error.set(this.INVALID_PASSWORD);
  return;
}
```
The registration flow should validate in this order:
```ts
if (!this.isValidEmail(email)) {
  this.display_error.set(this.INVALID_EMAIL);
  return;
}
if (password.length < 8) {
  this.display_error.set(this.INVALID_PASSWORD);
  return;
}
```

```
This prevents invalid short passwords from calling `/auth/signup`.
## 3. Disable Register until the form is valid
### File: `login.component.ts`
Add a computed signal:
```ts
import { computed } from '@angular/core';
```
Then:
```ts
canRegister = computed(() => {
  const email = this.user.value ?? '';
  const password = this.pass.value ?? '';
  return this.isValidEmail(email) && password.length >= 8;
});
```
Because Angular `FormControl` values are not automatically signals, the more reliable approach is to expose the validation directly in the template.
### File: `login.component.html`
Change:
```html
<button class="btn btn-secondary" (click)="register()">Register</button>
```
to:
```html
<button
  class="btn btn-secondary"
  [disabled]="!isValidEmail(user.value ?? '') || (pass.value ?? '').length < 8"
  (click)="register()"
>
  Register
</button>
```
Since `isValidEmail()` is currently private, make it public by removing `private`:
```ts
isValidEmail(email: string): boolean {
  return /^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(email);
}
```
You can also show the password requirement:
```

```html
<p class="hint">Password must be at least 8 characters.</p>
```
Add the backend/custom error to the template:
```html
<div *ngIf="backend_error()" class="alert alert-error">
  {{ backend_error() }}
</div>
```
Since `display_error` becomes a signal, update existing template checks from:
```html
*ngIf="WRONG_PASSWORD === display_error"
```
to:
```html
*ngIf="WRONG_PASSWORD === display_error()"
```
Do the same for the other error blocks:
```html
<div *ngIf="USERNAME_TAKEN === display_error()" class="alert alert-error">
  Email already registered
</div>
<div *ngIf="INVALID_EMAIL === display_error()" class="alert alert-error">
  Enter a valid email address
</div>
<div *ngIf="INVALID_PASSWORD === display_error()" class="alert alert-error">
  Password must be at least 8 characters
</div>
```
For generic errors:
```html
<div
  *ngIf="OTHER_ERROR === display_error() && !backend_error()"
  class="alert alert-error"
>
  Something went wrong, try again
</div>
```
This avoids displaying both the backend error and the generic error at the same time.
## 4. Show profile save errors

### File: `profile.component.ts`
You already have:
```ts
profile_saved = signal(false);
```
Add:
```ts
profile_error = signal('');
```
Update `saveProfile()`:
```ts
saveProfile() {
  this.profile_saved.set(false);
  this.profile_error.set('');
  this.http_client
    .put(`${API_BASE}/profile`, this.profile_form.getRawValue())
    .subscribe({
      next: () => this.profile_saved.set(true),
      error: (err) =>
        this.profile_error.set(
          err?.error?.error ?? 'Could not save profile',
        ),
    });
}
```
### File: `profile.component.html`
Add this after the save button:
```html
<div *ngIf="profile_error()" class="alert alert-error">
  {{ profile_error() }}
</div>
```
The existing success message can remain:
```html
<div *ngIf="profile_saved()" class="alert alert-success">
  Saved.
</div>
```
## 5. Hide the Resumes section for recruiters
### File: `profile.component.html`

Wrap the entire resumes card in an `*ngIf`:
```html
<div class="card resumes-card" *ngIf="user()?.role !== 'recruiter'">
  ...
</div>
```
This hides the section visually.
### File: `profile.component.ts`
You should also avoid requesting resumes for recruiters, because the backend may reject that request. Change:
```ts
ngOnInit() {
  this.refreshResumes();
  this.http_client.get<{ profile: ProfileData }>(`${API_BASE}/profile`)
```
to:
```ts
ngOnInit() {
  if (this.user()?.role !== 'recruiter') {
    this.refreshResumes();
  }
  this.http_client.get<{ profile: ProfileData }>(`${API_BASE}/profile`)
```
## Summary of required changes
| File | Changes |
|---|---|
| `login.component.ts` | Use signals for error state, validate password length, display backend 400 messages |
| `login.component.html` | Call signals with `()`, disable Register when invalid, show password/backend errors |
| `profile.component.ts` | Add profile save error signal and error handler; skip resume loading for recruiters |
| `profile.component.html` | Display profile save error; hide Resumes card for recruiters |

The role selection, role submission, and post-signup redirects are already implemented. These changes address the remaining missing behaviors.

AI-Suggested Content : The list of files and changes, the code snippets for each, the use of err.error?.error to read the backend's 400 message, and the summary table of changes per file.

Validation : I compared the suggestions with my assignment in the #112 review and with the actual code in the repo before applying anything. Some of the suggested code was not correct

as written. The backend_error message was never cleared between attempts. The template syntax and the recruiter role check also depended on how my project works, so I had to adapt them. I applied the changes myself and adjusted them to fit the project. Unit tests (HttpTestingController for 401, 409 and 400) and acceptance test #88 AT-4 are still pending.

Decision : Accepted and modified. I used the AI's file list and overall approach, but wrote and corrected the code myself.

Reflection : The AI helped me see which files to change and gave me a starting point, but it only gave suggestions and I had to check each change against my own code.

Responsible Person : Zohair Ameeri

**Task #008 : Understand Git branch output and confirm which branch I am on**

Purpose of AI use : To understand the output of git branch -a and git branch in my terminal and to confirm whether I was on the main branch before starting my work. The AI only explained the output. It did not write or change any code.

Chat Link or Prompt : [paste chat link]. Prompt: pasted my terminal output (a fetch that created the new remote branch origin/ReadME-and-Minutes-Changes, then git branch -a and git branch) and asked what it means and whether I am not on the main branch.

Response : The AI explained that I was on main, because main is the branch marked with the asterisk. It explained that git branch -a lists local branches plus the remote branches under remotes/origin/ (the branches on GitHub), and that git branch lists only my local branches. It confirmed that I have three local branches (feature/task-74-registration, main and recruiter-postings-ui) and that main is the active one.

AI-Suggested Content : An explanation of the terminal output and the difference between git branch -a and git branch. No code or commands were suggested.

Validation : I compared the explanation with my own terminal output. main is the branch marked with the asterisk, and the local and remote branch lists match what the AI described.

Decision : Accepted. I now know I am on main, which is what I need before branching off for my follow-up PR after #112 merges.

Reflection : The AI gave a quick, clear explanation of the output, which saved me from searching for it. It was mostly accurate, but I still checked it against my terminal, and it is no replacement for knowing the basic Git commands.

Responsible Person : Zohair Ameeri

**Task #009 : Understand how git add . works**

Purpose of AI use : To understand what the git add . command does and how it fits into the commit workflow before committing my changes. The AI only explained the command. It did not write or change any code.

Chat Link or Prompt : [paste chat link]. Prompt: "how git add . work".

Response : The AI explained that git add . stages all changes in the current directory so they are ready for the next commit. git add moves changes into the staging area, and the period means the current folder and everything inside it. It stages new files, modified files and deleted files. After staging, the next step is usually git commit.

AI-Suggested Content : An explanation of the command using a shipping box analogy (staging is putting items in the box, committing is sealing it). No code changes were suggested.

Validation : [Describe how you checked it, e.g. ran git status before and after git add . and confirmed that my new, modified and deleted files moved to the staged section.] The explanation did not mention that files listed in .gitignore are not staged, and that git add . stages everything under the current folder, which can include files I did not mean to commit. I need to check git status before committing.

Decision : Accepted. I used git add . to stage my changes, then reviewed them before running git commit.

Reflection : The AI gave a short, clear explanation that was enough to understand the command. It left out a few details, such as .gitignore and the risk of staging unwanted files, so I still check git status before committing.

Responsible Person : Zohair Ameeri

**Task #0010: Write GitHub pull request description for login/register error handling and profile fixes (follow-up to #112)**

Purpose of AI use : To generate a properly formatted GitHub pull request description (title, summary, changes, pending items) from the list of changes I implemented, so it displays well in the GitHub preview. The AI wrote the description text only. It did not write or change any code.

Chat Link or Prompt : Prompt: gave a changelog of my changes (zoneless error signals and backend 400 handling, client-side password validation with inline feedback, profile update error feedback, recruiter Resumes restrictions) and asked for a PR description with a proper title in GitHub comment format, using my username ZA-Error. Later shared the PR #112 page with the review assignments and asked for a version that only describes what I did, lists unit tests and acceptance test #88 AT-4 as pending, and credits Claude AI as co-author.

Response : A PR description in GitHub-flavored Markdown with a conventional-commit style title, a summary linking it as a follow-up to #112 and the review assignment (#74, #75), my changes grouped into Auth and Profile, a Pending section (unit tests with HttpTestingController for 401, 409 and 400, and acceptance test #88 AT-4), an AI assistance note, and an author line crediting me and Claude AI. The AI also explained that the description does not auto-close any issues because it has no closing keyword like "Closes".

AI-Suggested Content : The PR title, the structure and formatting, the wording of each change, the "Follows #74 and #75" references, the Pending section, and the AI credit line.

Validation : I compared the final description with the changes I actually made and with my assignment in the #112 review. The AI's first draft was written before it saw the PR page, and it was written as if for #112 itself. After I shared the page, it turned out #112 belongs to another team member and my work is a separate follow-up PR, so the description was rewritten. The AI also added things I did not ask for (a test plan, notes and a Related section), and I asked it to trim the description to only what I did and what is still pending. I checked that the final version only lists work I did and that the references do not auto-close any issues.

Decision : Accepted and modified. I used the AI's structure and formatting, trimmed the description to what I implemented, and marked the unit tests and acceptance test #88 AT-4 as pending.

Reflection : The AI saved time on formatting and structure, but its first drafts assumed things without seeing the PR page and included content I did not ask for. I had to review each draft against my real changes and the review assignment, so I treated it as a drafting aid and checked the facts myself.

Responsible Person : Zohair Ameeri